

REACT BASICS

1. Objective

The objective of this assignment was to develop a small React component that consumes data from a REST API endpoint and handles different application states such as **loading, successful data retrieval, and errors**.

The component was developed using React and an AI coding assistant was used to assist with the implementation. The exercise also focused on understanding how React manages state and performs API requests.

The main objectives were:

- Create a reusable React component.
- Consume data from a REST API.
- Display a loading state while data is being fetched.
- Display an error message when the API request fails.
- Display the retrieved data after a successful request.
- Use an AI coding assistant to assist with development.
- Document the prompt used with the AI coding assistant.

2. Introduction

React is a JavaScript library used for building user interfaces. React applications are composed of reusable components that manage their own data and presentation.

Modern React applications frequently communicate with backend services through **REST APIs**. A REST API allows a frontend application to request data from a server and use that data to dynamically update the user interface.

When consuming an API, the application should account for different states:

1. **Loading state** – displayed while the API request is in progress.
2. **Success state** – displayed when the API returns data successfully.
3. **Error state** – displayed when the API request fails.

Handling these states improves the user experience and prevents the application from appearing unresponsive during network operations.

3. React Component Implementation

A small React component was created to retrieve and display data from a REST API endpoint.

The component uses React's `useState` hook to maintain the application state and the `useEffect` hook to perform the API request when the component is loaded.

Component Code

```
import { useEffect, useState } from "react";

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        if (!response.ok) {
          throw new Error("Failed to fetch users");
        }
        return response.json();
      })
      .then((data) => {
        setUsers(data);
        setLoading(false);
      })
      .catch((err) => {
        setError(err.message);
        setLoading(false);
      });
  }, []);

  if (loading) {
    return <p>Loading users...</p>;
  }
}
```

```

    }

    if (error) {
      return <p>Error: {error}</p>;
    }

    return (
      <div>
        <h2>User List</h2>

        {users.map((user) => (
          <div key={user.id}>
            <h3>{user.name}</h3>
            <p>{user.email}</p>
          </div>
        ))}
      </div>
    );
  }
}

export default UserList;

```

4. Component Explanation

useState

The useState hook is used to store the component's current state.

Three state variables are used:

```

const [users, setUsers] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState("");

```

- users stores the data returned by the API.
- loading determines whether the API request is still in progress.
- error stores an error message if the request fails.

useEffect

The `useEffect` hook is used to execute the API request when the component is initially rendered.

```
useEffect(() => {  
  fetch("https://jsonplaceholder.typicode.com/users")  
})
```

The empty dependency array ensures that the request is executed when the component is mounted.

API Request

The `fetch()` function sends a request to the REST API endpoint.

```
fetch("https://jsonplaceholder.typicode.com/users")
```

The response is first checked using:

```
if (!response.ok) {  
  throw new Error("Failed to fetch users");  
}
```

If the response is successful, it is converted into JSON format and stored in the `users` state.

5. Loading, Error and Success States

The component handles three primary states.

5.1 Loading State

Initially, `loading` is set to `true`.

```
if (loading) {  
  return <p>Loading users...</p>;  
}
```

While the API request is being processed, the user sees a loading message.

This provides feedback that the application is actively retrieving information.

5.2 Error State

If the API request fails, the error is stored in the error state.

```
.catch((err) => {  
  setError(err.message);  
  setLoading(false);  
});
```

The component then displays:

```
if (error) {  
  return <p>Error: {error}</p>;  
}
```

This prevents the application from failing silently and informs the user that the requested data could not be retrieved.

5.3 Success State

When the API request is successful, the returned data is stored in the users state.

The component uses the `map()` function to display each user:

```
{users.map((user) => (  
  <div key={user.id}>  
    <h3>{user.name}</h3>  
    <p>{user.email}</p>  
  </div>  
))}
```

Each user is displayed with their name and email address.

6. AI Coding Assistant Prompt

An AI coding assistant was used to assist with developing the React component.

The following prompt was used:

Create a React functional component that consumes a REST API using fetch.

Use the JSONPlaceholder users endpoint:

<https://jsonplaceholder.typicode.com/users>

Requirements:

1. Use React useState and useEffect.
2. Fetch the users when the component loads.
3. Display "Loading users..." while the request is in progress.
4. Display an error message if the API request fails.
5. Display the user's name and email when the request succeeds.
6. Handle non-successful HTTP responses.
7. Keep the component simple and reusable.
8. Provide the complete React component code.

The AI-generated implementation was reviewed and used as assistance during development. The generated code was then understood, tested, and verified rather than being used without review.

7. Testing and Verification

The React component was tested to verify the different application states.

Test Case 1 – Successful API Request

When the API request succeeds, the component retrieves the user data and displays the list of users.

Expected behavior:

The loading message disappears and the retrieved user information is displayed.

Test Case 2 – Loading State

When the component initially loads, the API request takes some time to complete.

Expected behavior:

The message "**Loading users...**" is displayed while the request is in progress.

Test Case 3 – API Error

If the API request fails or returns an unsuccessful HTTP response, the error state is triggered.

Expected behavior:

An appropriate error message is displayed instead of the user list.

8. Key Learnings

The assignment provided practical understanding of how React applications communicate with REST APIs.

The following concepts were covered:

- React functional components.
- `useState` for managing component state.
- `useEffect` for performing side effects.
- JavaScript `fetch()` for making API requests.
- JSON response handling.
- Conditional rendering.
- Loading and error states.
- Rendering API data using `map()`.
- Handling unsuccessful HTTP responses.
- Using AI coding assistants during application development.

The exercise also demonstrated that AI coding assistants can accelerate development by generating initial implementations and suggesting solutions. However, the generated code should always be reviewed, understood, tested, and modified according to project requirements.

9. Conclusion

The React Basics assignment successfully demonstrated how a React component can consume data from a REST API and manage different application states.

The component uses `useState` to manage users, loading, and error information, while `useEffect` is used to perform the API request when the component loads. Conditional

rendering provides appropriate feedback during loading, successful data retrieval, and API failures.

The use of an AI coding assistant also demonstrated how AI tools can support frontend development by helping generate and improve code. The implementation was reviewed and tested to ensure that it met the specified requirements.

Overall, the assignment provided practical experience with **React components, REST API integration, state management, asynchronous operations, error handling, and AI-assisted development.**

ANNAPOORNA V

69312